



SORBONNE UNIVERSITÉ
MASTER ANDROIDE

**Analyse de l'évolution des codes Python
dans des plateformes d'apprentissage**

UE de projet M1

Victor ROQUE GOMES Lucke MAO Victor YE

Encadrants : Sébastien Jolivet, Sébastien Lallé et Amel Yessad

Table des matières

1	Introduction	1
2	État de l’art	2
2.1	Représentation de code par AST	2
2.2	Distance d’édition sur les arbres : algorithme de Zhang-Shasha	3
2.3	Analyse de code étudiant	3
2.4	Analyse de trajectoires d’apprentissage	4
2.5	Feedback automatique dans les environnements d’apprentissage	4
3	Contribution	6
3.1	Pipeline général	6
3.2	Préparation des données AlgoPython	6
3.3	Analyse AST Python	7
3.4	Distance structurelle entre programmes	7
3.5	Détection d’erreurs de code	7
3.6	Génération de datasets de transitions	7
3.7	Cas d’étude	8
3.7.1	Cas 1 — Comparaison brute des programmes	8
3.7.2	Cas 2 — Génération de commentaires pédagogiques	8
3.8	Classification des profils étudiants	8
3.8.1	Méthode de classification	9
3.9	Visualisations et analyses statistiques	10
3.10	Structure du projet	11
3.10.1	Organisation des modules	11
3.10.2	Organisation des fonctions du projet	12
3.10.3	Tests unitaires	13
4	Résultats d’analyse	14
4.1	Distribution des profils étudiants	14
4.2	Distance ZSS moyenne selon le numéro de tentative	15
4.3	Progression moyenne selon le numéro de tentative	16
4.4	Synthèse	17
5	Conclusion	18
5.1	Limites du projet	18
5.2	Perspectives	19
A	Cahier des charges	21

Chapitre 1

Introduction

Ce projet s'inscrit dans le cadre des travaux de recherche du master [AI2D](#) (Algorithmes, Intelligence Artificielle, Interactions et Décision) de Sorbonne Université. Il vise à analyser l'évolution des codes Python produits par des étudiants au sein de plateformes d'apprentissage de la programmation.

Le contexte applicatif repose sur plusieurs environnements d'apprentissage de la programmation, notamment des jeux sérieux et des exercices, au sein desquels l'étudiant produit plusieurs codes successifs pour résoudre un problème. Parmi ces environnements, on peut citer [SPY](#), [AlgoPython](#) et [Pirates](#). L'enjeu est d'identifier et de caractériser les évolutions du code initial vers le code final soumis par les étudiants, et d'exprimer ces modifications de manière lisible et pertinente pour un enseignant d'informatique.

Ce projet a été réalisé sous la supervision de Sébastien Jolivet, Sébastien Lallé et Amel Yes-sad. Le dépôt git du projet est accessible à l'adresse suivante : github.com/Driw0x/Projet-AI2D.

Chapitre 2

État de l’art

Ce chapitre présente les concepts et outils existants en lien avec l’analyse des évolutions de code source dans un contexte pédagogique.

2.1 Représentation de code par AST

Un *Abstract Syntax Tree* (AST) est une représentation arborescente de la structure syntaxique d’un programme. Chaque nœud de l’arbre représente une construction du langage (expression, instruction, opérateur, etc.). En Python, le module standard `ast` permet de générer et de manipuler ces arbres facilement.

L’utilisation des AST pour comparer deux versions d’un programme est au cœur de nombreux travaux sur la détection de plagiat, la rétroaction automatique et le suivi de l’apprentissage.

Pourquoi utiliser les AST pour comparer deux programmes ? Comparer deux programmes directement au niveau du texte brut — par exemple avec un *diff* ligne à ligne — présente de nombreuses limites : deux programmes sémantiquement équivalents peuvent différer par leur mise en forme, leurs commentaires ou l’ordre de certaines instructions. À l’inverse, une modification mineure du texte peut masquer un changement structurel profond.

Les AST s’affranchissent de ces ambiguïtés en représentant uniquement la structure logique du programme, indépendamment des détails de style. Comparer deux AST revient donc à comparer la *structure* des deux programmes : quels constructs sont présents, dans quel ordre, et avec quelles relations hiérarchiques. Cette représentation permet d’identifier précisément quelle construction a été ajoutée, supprimée ou modifiée — par exemple l’ajout d’une boucle `for` ou la modification d’une condition `if` — ce qui est directement interprétable d’un point de vue pédagogique.

Usages pédagogiques des AST. Au-delà de la simple comparaison structurelle, plusieurs travaux ont exploré les AST comme support d’analyse pédagogique. Piech et al. ont utilisé des représentations arborescentes du code pour identifier des erreurs conceptuelles récurrentes chez les étudiants débutants en programmation [1]. D’autres travaux s’en servent pour regrouper automatiquement des soumissions similaires et ainsi réduire

la charge de correction manuelle des enseignants. Dans ce cadre, les AST permettent de dépasser la notion de « bonne réponse / mauvaise réponse » pour s'intéresser à *comment* l'étudiant a structuré sa solution, indépendamment de son exactitude.

2.2 Distance d'édition sur les arbres : algorithme de Zhang-Shasha

La distance d'édition entre deux arbres (*Tree Edit Distance*, TED) est une mesure classique permettant de quantifier la dissimilarité entre deux AST. Elle est définie comme le coût minimal d'une séquence d'opérations élémentaires — insertion, suppression, et substitution de nœuds — permettant de transformer un arbre en un autre.

L'algorithme de **Zhang-Shasha** [2] est l'un des algorithmes de référence pour calculer cette distance de manière exacte en temps polynomial. Il est particulièrement adapté à la comparaison de petits arbres syntaxiques comme ceux produits par des programmes Python d'exercices scolaires.

La librairie [ast-error-detection](#), utilisée dans ce projet, s'appuie sur cet algorithme : à partir de deux codes Python, elle les convertit en AST et renvoie la liste des opérations d'édition nécessaires pour passer de l'un à l'autre.

Par exemple, pour transformer un code erroné en sa version correcte (changement du nombre d'itérations d'une boucle `for` et indentation de son corps), il faut exactement deux opérations selon cet algorithme.

2.3 Analyse de code étudiant

L'analyse automatique du code produit par des étudiants est un champ de recherche actif en informatique éducative. Les travaux existants poursuivent généralement deux objectifs complémentaires : détecter les erreurs et incompréhensions, et comprendre les stratégies de résolution adoptées.

Détection d'erreurs et de misconceptions. Des outils comme *Overcode* (Glassman et al. [3]) regroupent les soumissions d'étudiants par similarité structurelle afin d'aider les enseignants à identifier les erreurs les plus fréquentes à l'échelle d'une classe. Ces approches reposent souvent sur une normalisation du code (renommage des variables, simplification des expressions) avant comparaison. D'autres travaux utilisent les AST pour détecter automatiquement des *misconceptions* — c'est-à-dire des erreurs traduisant une incompréhension conceptuelle — plutôt que de simples fautes de syntaxe.

Analyse des soumissions successives. Plusieurs études se sont intéressées non pas à une soumission isolée, mais à la séquence complète des tentatives d'un étudiant. Price et al. [4] ont montré que l'analyse des *snapshots* de code intermédiaires permet de mieux comprendre le processus de résolution que la seule soumission finale. Cette perspective est directement en lien avec notre approche, qui exploite l'ensemble des soumissions successives disponibles dans AlgoPython pour caractériser la trajectoire de chaque étudiant.

2.4 Analyse de trajectoires d'apprentissage

Des travaux en *Learning Analytics* s'intéressent à la modélisation des trajectoires d'apprentissage, c'est-à-dire aux séquences d'états traversés par un étudiant au cours de la résolution d'un exercice. Des approches de clustering (k-means, classification hiérarchique) ont été appliquées pour identifier des profils d'étudiants à partir de ces séquences [5].

Clustering de trajectoires de programmation. Appliquer des algorithmes de clustering à des trajectoires de code pose des défis spécifiques : les séquences sont de longueur variable, les soumissions peuvent être syntaxiquement invalides, et la notion de « distance » entre deux trajectoires est moins immédiate que pour des données numériques. Des approches comme DTW (*Dynamic Time Warping*) ou la vectorisation des états intermédiaires ont été proposées pour contourner ces difficultés. Dans notre projet, chaque état d'une trajectoire est représenté par un vecteur de caractéristiques (distance ZSS, progression, statut de réussite) permettant l'application de méthodes de clustering standard.

Trajectoires chez les étudiants qui apprennent la programmation. Dans notre contexte, nous nous intéressons plus spécifiquement aux trajectoires des étudiants qui apprennent à programmer. Contrairement à d'autres domaines d'apprentissage, la programmation produit des artefacts concrets et versionés : à chaque soumission, l'étudiant laisse une trace précise de son état de compréhension. L'ensemble de ces soumissions pour un exercice donné constitue une *trajectoire de résolution*, qui reflète la stratégie adoptée par l'étudiant pour passer d'un code incorrect ou incomplet à une solution valide.

Ces trajectoires permettent d'observer des comportements variés : certains étudiants progressent par petits ajustements successifs, d'autres procèdent par essais-erreurs massifs ou restent bloqués sur les mêmes erreurs. L'analyse de ces patterns ouvre la voie à une meilleure compréhension des difficultés rencontrées et à la proposition d'une aide pédagogique ciblée de la part des enseignants.

2.5 Feedback automatique dans les environnements d'apprentissage

De nombreuses plateformes d'apprentissage de la programmation intègrent des mécanismes de feedback automatique pour guider les étudiants. On distingue généralement deux niveaux de feedback :

Feedback syntaxique et fonctionnel indique si le programme compile et passe les tests. C'est le niveau minimal fourni par la quasi-totalité des exercices, dont AlgoPython.

Feedback sémantique et pédagogique explique *pourquoi* le code est incorrect et suggère une direction de correction. Ce niveau est plus rare et constitue un objectif de recherche actif.

Des systèmes comme *Automated Feedback Generation* (Singh et al. [6]) utilisent la distance entre l'AST de l'étudiant et celui d'une solution de référence pour générer automa-

tiquement des suggestions de correction minimales. D'autres approches s'appuient sur des modèles de langage pour produire des explications en langage naturel.

Dans le cas d'AlgoPython, un feedback est déjà fourni à chaque tentative. Notre travail se situe à un niveau différent : plutôt que de générer un feedback supplémentaire, nous cherchons à *caractériser* comment les étudiants réagissent aux feedbacks existants, en analysant l'évolution structurelle de leur code entre chaque soumission.

Chapitre 3

Contribution

Ce chapitre décrit les fonctionnalités que nous avons conçues et implémentées. Le projet vise à répondre à des questions telles que : un étudiant progresse-t-il réellement entre deux tentatives ? Corrige-t-il par petits ajustements ou par grosses restructurations ? Quels types d'erreurs sont les plus fréquents ? Quels profils d'apprentissage émergent ?

3.1 Pipeline général

Le pipeline du projet suit les étapes suivantes :

1. **Données AlgoPython** : chargement des données brutes au format JSON.
2. **Nettoyage / Normalisation** : transformation en DataFrames pandas exploitables.
3. **Extraction des trajectoires** : reconstruction des séquences de tentatives par étudiant et par exercice.
4. **Comparaison $t \rightarrow t + 1$** : analyse de chaque paire de soumissions consécutives.
5. **Analyse AST** : parsing et comparaison structurelle des programmes.
6. **Détection d'erreurs** : identification automatique des types d'erreurs.
7. **Mesure de progression** : quantification des évolutions entre soumissions.
8. **Classification des profils** : regroupement des étudiants par stratégie d'apprentissage.

3.2 Préparation des données AlgoPython

Le pipeline transforme les données brutes issues d'AlgoPython en DataFrames pandas exploitables. Les opérations effectuées sont :

- explosion des structures JSON imbriquées ;
- extraction des classes, comptes et trajectoires ;
- normalisation des tentatives ;
- nettoyage des données ;
- binarisation des statuts de réussite ($\text{ok} \rightarrow 1$, sinon 0).

La fonction principale de cette étape est `AlgoPython_data(df)`.

3.3 Analyse AST Python

Le projet utilise le module `ast` de Python pour :

- parser les programmes soumis par les étudiants ;
- construire leurs arbres syntaxiques abstraits ;
- comparer les structures de code entre deux soumissions ;
- préparer les vecteurs de distance AST.

Les fonctions principales sont `code_to_ast(code)` et `ast_dump(tree)`.

3.4 Distance structurelle entre programmes

Le projet mesure les différences entre deux programmes via trois composantes :

- la distance de Zhang-Shasha (nombre minimal d’opérations AST) ;
- la liste détaillée des opérations de transformation ;
- la comparaison directe $t \rightarrow t + 1$ entre soumissions consécutives.

Les fonctions principales sont `compare_transition(...)` et `distance(...)`.

3.5 Détection d’erreurs de code

Le projet s’appuie sur la librairie `ast_error_detection` pour identifier automatiquement les erreurs présentes dans le code d’un étudiant. Les types d’erreurs détectées incluent :

- erreurs d’appels de fonctions ;
- erreurs de boucles ;
- erreurs de conditions ;
- erreurs d’assignation ;
- erreurs d’opérations.

Les fonctions principales sont `primary_code_error_two_prog(p1, p2)`, qui compare deux programmes, et `prog_vs_answer(p1, list_answer)`, qui compare un programme à une liste de solutions attendues.

3.6 Génération de datasets de transitions

Le pipeline construit automatiquement un dataset de transitions de la forme tentative $t \rightarrow$ tentative $t + 1$. Chaque transition est enrichie des attributs suivants :

- distance AST entre les deux soumissions ;
- indicateur de progression ;
- temps écoulé entre les deux tentatives ;
- statut de réussite ;

- erreurs détectées ;
- comparaison aux solutions attendues.

La fonction principale est `build_transition_dataset(...)`.

3.7 Cas d'étude

3.7.1 Cas 1 — Comparaison brute des programmes

Le premier cas d'étude produit un dataset structuré contenant, pour chaque paire de soumissions consécutives :

- la distance ZSS (Zhang-Shasha) ;
- les opérations AST de transformation ;
- les erreurs principales détectées ;
- les typologies d'erreurs ;
- les codes t et $t + 1$.

La fonction associée est `cas1(...)`.

3.7.2 Cas 2 — Génération de commentaires pédagogiques

Le second cas d'étude transforme automatiquement les erreurs détectées en phrases interprétables par un enseignant. Par exemple :

- « **Ajout d'une boucle for** » généré lors de l'insertion d'un nœud `For`
- « **Suppression d'un appel à la fonction** » généré lors de la suppression d'un nœud `Call`
- « **Changement de constante** » généré lors de la substitution d'un nœud `Constant`

La fonction associée est `cas2(...)`.

3.8 Classification des profils étudiants

Le projet génère des profils dynamiques pour chaque étudiant à partir des trajectoires de résolution observées dans les données AlgoPython.

Ces profils sont construits à partir de plusieurs dimensions :

- la progression entre les tentatives ;
- le temps passé sur les exercices ;
- le nombre de tentatives ;
- l'importance des restructurations du code ;
- le taux de réussite des exercices.

L'objectif est d'identifier différentes stratégies de résolution et différents comportements d'apprentissage observés chez les étudiants.

Les expérimentations menées sur les données AlgoPython ont permis d'identifier plusieurs profils comportementaux récurrents :

expert_progressif étudiant qui améliore régulièrement son code et atteint rapidement une solution correcte.

reviseur_methodique étudiant qui progresse par petites corrections successives et ajuste progressivement son programme.

explorateur_chaotique étudiant dont les modifications sont importantes, fréquentes et peu structurées.

bloque étudiant qui présente peu d'évolution entre les tentatives et rencontre des difficultés persistantes.

perseverant_lent étudiant nécessitant un grand nombre de tentatives avant d'atteindre une solution correcte.

stagne_mais_corrige étudiant semblant peu progresser au départ mais parvenant finalement à corriger son programme.

restructurateur_lent étudiant effectuant des modifications importantes mais progressant lentement vers une solution correcte.

fragile étudiant présentant une progression instable avec des phases d'amélioration puis de régression.

gros_restructurateur_productif étudiant réalisant de fortes restructurations du code tout en améliorant progressivement sa solution.

apprenant_regulier étudiant avançant de manière stable avec une progression relativement constante.

petits_pas_nombreux étudiant effectuant un grand nombre de petites corrections successives avant d'atteindre une solution correcte.

en_difficulte étudiant rencontrant des difficultés persistantes avec une progression très limitée malgré plusieurs tentatives.

Les fonctions principales associées à cette partie sont `build_user_classification(...)` et `build_user_exercice_classification(...)`.

3.8.1 Méthode de classification

La classification des profils repose sur plusieurs indicateurs extraits des trajectoires de résolution et des transitions entre tentatives.

Pour chaque étudiant, le pipeline calcule notamment :

- le nombre total de tentatives ;
- le taux de réussite des exercices ;
- la distance moyenne de Zhang-Shasha entre tentatives ;
- la progression vers les solutions attendues ;
- le temps moyen passé sur les exercices ;
- l'importance des restructurations de code.

Ces indicateurs permettent d'estimer la manière dont un étudiant corrige et fait évoluer son programme au cours de la résolution d'un exercice.

Les profils sont actuellement générés à partir d'heuristiques définies manuellement. Par exemple :

- un étudiant ayant peu de tentatives, de faibles distances AST et un fort taux de réussite sera associé au profil `expert_progressif` ;
- un étudiant réalisant de nombreuses petites corrections successives sera plutôt associé au profil `reviseur_methodique` ;
- un étudiant effectuant de fortes restructurations de code avec des variations importantes de distance AST sera davantage associé au profil `explorateur_chaotique` ;
- un étudiant présentant peu d'évolution entre les tentatives et un faible taux de réussite pourra être classé dans le profil `bloque` ;
- un étudiant nécessitant un grand nombre de tentatives avant d'atteindre une solution correcte sera associé au profil `perseverant_lent`.

Les autres profils correspondent à des comportements intermédiaires ou plus spécifiques observés lors des expérimentations sur les trajectoires de résolution.

Cette approche permet d'obtenir une première interprétation pédagogique des trajectoires de programmation observées dans les données AlgoPython.

Cependant, cette classification reste fondée sur des règles heuristiques et ne constitue pas encore un modèle d'apprentissage automatique supervisé ou non supervisé.

3.9 Visualisations et analyses statistiques

Le projet intègre également plusieurs outils de visualisation permettant d'explorer les trajectoires d'apprentissage et les comportements des étudiants.

Les graphiques générés permettent notamment d'étudier :

- la distribution des distances de Zhang-Shasha ;
- la progression des étudiants vers les solutions attendues ;
- la fréquence des erreurs détectées ;
- le nombre moyen de tentatives par exercice ;
- la répartition des profils étudiants ;
- les variations de distance entre tentatives successives.

Ces visualisations offrent une interprétation plus intuitive des données et permettent de mettre en évidence plusieurs comportements pédagogiques :

- étudiants progressant par petites corrections successives ;
- étudiants effectuant de fortes restructurations de code ;
- exercices provoquant des erreurs récurrentes ;
- différences de stratégies de résolution entre étudiants.

Les fonctions de génération graphique sont regroupées dans le module `visualisations.py`.

3.10 Structure du projet

Le projet est organisé de la façon suivante :

```
Projet-AI2D/
|
+-- main.py
+-- requirements.txt
+-- README.md
+-- .gitignore
|
+-- data/
|   +-- data.json
|   +-- solutions.json
|   +-- cas1.json
|   +-- cas2.json
|   +-- pre_calcul.json
|
+-- models/
|   +-- differences_tag.py
|   +-- evolution_code.py
|
+-- tools/
|   +-- __init__.py
|   +-- ast_tools.py
|   +-- cas.py
|   +-- classification.py
|   +-- comparaison.py
|   +-- dataset_builder.py
|   +-- io_tools.py
|   +-- visualisations.py
|
+-- tests/
|   +-- __init__.py
|   +-- all.py
|   +-- conftest.py
|   +-- test_ast_tools.py
|   +-- test_cas.py
|   +-- test_classification.py
|   +-- test_comparaison.py
|   +-- test_dataset_builder.py
|   +-- test_io_tools.py
```

3.10.1 Organisation des modules

Le dossier `tools/` contient les principales briques fonctionnelles du pipeline.

Module	Rôle
<code>io_tools.py</code>	Lecture, nettoyage et transformation des données AlgoPython
<code>ast_tools.py</code>	Parsing Python, génération et manipulation des AST
<code>comparaison.py</code>	Comparaison de programmes, calcul des distances AST et détection d'erreurs
<code>dataset_builder.py</code>	Construction des datasets de transitions entre tentatives
<code>cas.py</code>	Génération des cas d'étude <code>cas1</code> et <code>cas2</code>
<code>classification.py</code>	Génération des profils étudiants
<code>visualisations.py</code>	Création de graphiques et analyses statistiques

Le dossier `models/` contient les structures de données utilisées pour représenter les différences entre programmes et les évolutions de code.

Le dossier `tests/` contient les tests unitaires du projet. Chaque module possède un fichier de test dédié afin de garantir la robustesse des différentes parties du pipeline.

3.10.2 Organisation des fonctions du projet

Les fonctionnalités du pipeline sont réparties dans plusieurs modules spécialisés du dossier `tools/`.

tools/io_tools.py Contient les fonctions de chargement, nettoyage et transformation des données AlgoPython. Ce module prépare les structures pandas utilisées dans le reste du pipeline.

- `AlgoPython_data(df)` : préparation et nettoyage des données brutes.

tools/ast_tools.py Regroupe les fonctions liées à la manipulation des arbres syntaxiques abstraits (AST) Python.

- `code_to_ast(code)` : conversion d'un code Python en AST ;
- `ast_dump(tree)` : représentation textuelle d'un AST.

tools/comparaison.py Contient les fonctions de comparaison entre programmes et les calculs de distances syntaxiques.

- `compare_transition(...)` : comparaison entre deux soumissions ;
- `distance(...)` : calcul de la distance de Zhang-Shasha ;
- `primary_code_error_two_prog(p1, p2)` : détection d'erreurs entre deux programmes ;
- `prog_vs_answer(p1, list_answer)` : comparaison avec les solutions attendues.

tools/dataset_builder.py Responsable de la génération des datasets de transitions entre tentatives.

- `build_transition_dataset(...)` : construction du dataset de transitions.

tools/cas.py Regroupe les fonctions de génération des cas d'étude.

- `cas1(...)` : génération du dataset de comparaison brute ;
- `cas2(...)` : génération des commentaires pédagogiques.

tools/classification.py Contient les fonctions de classification des profils étudiants.

- `build_user_classification(...)` : classification globale des utilisateurs ;
- `build_user_exercice_classification(...)` : classification par utilisateur et par exercice.

tools/visualisations.py Contient les fonctions de génération des graphiques et analyses statistiques du projet.

3.10.3 Tests unitaires

Afin d'assurer la robustesse du pipeline, le projet intègre une suite de tests unitaires basée sur `pytest`.

Chaque module principal possède son propre fichier de test :

- `test_ast_tools.py`
- `test_comparaison.py`
- `test_dataset_builder.py`
- `test_classification.py`
- `test_cas.py`
- `test_io_tools.py`

Ces tests permettent notamment de vérifier :

- le parsing correct des AST ;
- la construction des datasets ;
- les calculs de distances ;
- les comparaisons entre programmes ;
- les classifications générées ;
- la stabilité du pipeline après modification du code.

L'exécution des tests peut être réalisée via :

```
pytest -v
```

ou directement via :

```
python tests/all.py
```

Chapitre 4

Résultats d'analyse

Ce chapitre présente les résultats obtenus en appliquant le pipeline développé sur les données issues de la plateforme AlgoPython.

4.1 Distribution des profils étudiants

```
===== Distribution des classes utilisateur =====
  | | | | | | | | | | classe  nb    %
0 | | | | | | | | | | bloque  281  33.06
1 | | | | | | | | | | explorateur_chaotique  237  27.88
2 | | | | | | | | | | stagne_mais_corrige  192  22.59
3 | | | | | | | | | | reviseur_methodique  42   4.94
4 | | | | | | | | | | restructurateur_lent  33   3.88
5 | | | | | | | | | | en_difficulte  18   2.12
6 | | | | | | | | | | expert_progressif  13   1.53
7 | | | | | | | | | | fragile  12   1.41
8 | | | | | | | | | | gros_restructurateur_productif  9   1.06
9 | | | | | | | | | | apprenant_regulier  7   0.82
10 | | | | | | | | | | petits_pas_nombreux  6   0.71

===== Distribution des classes utilisateur/exercice =====
  | | | | | | | | | | classe  nb    %
0 | | | | | | | | | | bloque  452  34.58
1 | | | | | | | | | | explorateur_chaotique  338  25.86
2 | | | | | | | | | | stagne_mais_corrige  279  21.35
3 | | | | | | | | | | reviseur_methodique  61   4.67
4 | | | | | | | | | | restructurateur_lent  49   3.75
5 | | | | | | | | | | en_difficulte  41   3.14
6 | | | | | | | | | | apprenant_regulier  23   1.76
7 | | | | | | | | | | expert_progressif  19   1.45
8 | | | | | | | | | | fragile  18   1.38
9 | | | | | | | | | | petits_pas_nombreux  16   1.22
10 | | | | | | | | | | gros_restructurateur_productif  11   0.84
```

FIGURE 4.1 – Distribution des classes utilisateur et utilisateur/exercice sur AlgoPython

La Figure 4.1 présente la répartition des profils identifiés par le pipeline, selon deux granularités : par utilisateur et par couple utilisateur/exercice.

Les résultats montrent que trois profils dominent largement l'ensemble des données :

bloque le profil le plus fréquent, représentant **33,06%** des utilisateurs (34,58% par exercice). Ce résultat indique qu'un tiers des étudiants présente peu d'évolution entre leurs tentatives et rencontre des difficultés persistantes à faire progresser leur code.

explorateur_chaotique le deuxième profil le plus représenté avec **27,88%** des utilisateurs (25,86% par exercice). Ces étudiants effectuent des modifications importantes et peu structurées, ce qui suggère une stratégie d'essais-erreurs sans direction claire.

stagne_mais_corrige troisième profil avec **22,59%** des utilisateurs (21,35% par exercice). Ces étudiants parviennent à corriger leur code malgré une progression apparemment lente ou irrégulière.

À l'inverse, les profils les plus positifs — **expert_progressif** (1,53%) et **reviseur_methodique** (4,94%) — restent minoritaires, ce qui suggère que la majorité des étudiants d'AlgoPython rencontre des difficultés significatives dans leur apprentissage de la programmation.

Les profils moins fréquents (**restructurateur_lent**, **en_difficulte**, **fragile**, **gros_restructurateur_p**, **apprenant_regulier**, **petits_pas_nombreux**) représentent chacun moins de 4% des cas et correspondent à des comportements plus spécifiques ou atypiques.

4.2 Distance ZSS moyenne selon le numéro de tentative

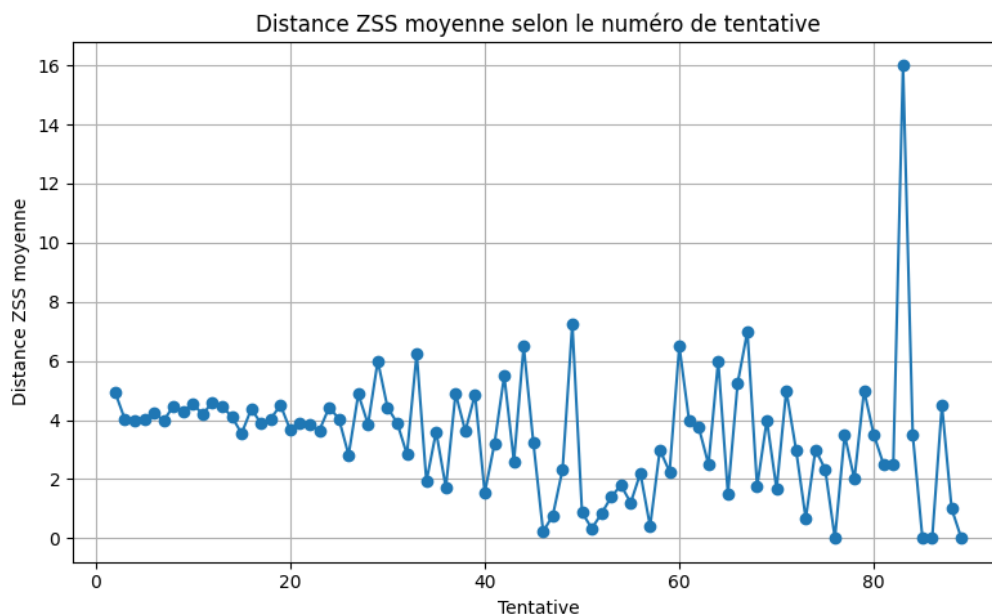


FIGURE 4.2 – Distance ZSS moyenne entre tentatives consécutives, selon le numéro de tentative

La Figure 4.2 représente l'évolution de la distance de Zhang-Shasha moyenne entre deux soumissions consécutives, en fonction du rang de la tentative.

Plusieurs observations peuvent être tirées de ce graphique :

- **Stabilité générale** : la distance ZSS oscille principalement entre 2 et 6, ce qui indique que la majorité des modifications d’une tentative à l’autre sont relativement locales — ajustement d’une valeur, ajout ou suppression d’une instruction.
- **Valeurs nulles** : plusieurs tentatives présentent une distance ZSS de 0, correspondant à des soumissions identiques à la précédente. Cela peut traduire une resoumission sans modification, voire une tentative de l’étudiant de vérifier que son code est accepté sans changement.
- **Pic anormal à la tentative ≈ 84** : un pic isolé à une distance de 16 est observable. Il correspond probablement à une restructuration massive du code par un étudiant ayant atteint un grand nombre de tentatives, ce qui est cohérent avec le profil `explorateur_chaotique` ou `gros_restructurateur_productif`.
- **Légère décroissance** : on observe une tendance à la baisse de la distance moyenne sur les premières tentatives (1 à 20), ce qui pourrait suggérer que les étudiants tendent à affiner progressivement leur code avec des modifications de plus en plus précises au fur et à mesure qu’ils avancent.

4.3 Progression moyenne selon le numéro de tentative

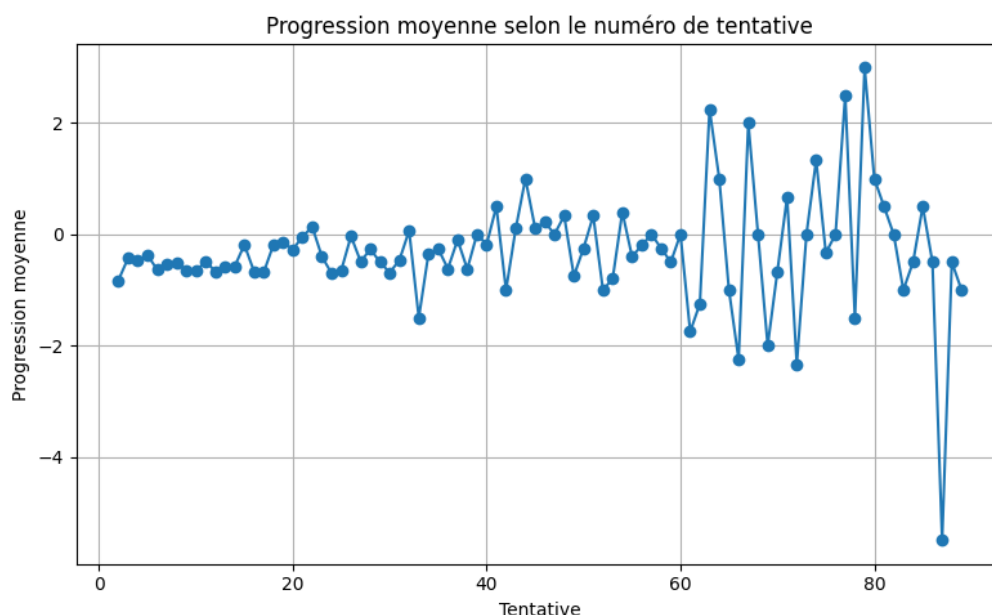


FIGURE 4.3 – Progression moyenne vers la solution selon le numéro de tentative

La Figure 4.3 illustre l’évolution de la progression moyenne entre tentatives consécutives, mesurée comme la variation de distance par rapport aux solutions attendues.

Les résultats mettent en évidence plusieurs comportements :

- **Progression globalement faible et oscillante** : les valeurs se situent principalement entre -1 et $+1$, indiquant que la majorité des transitions entre tentatives n’apportent pas d’amélioration significative vers la solution. Ce résultat est cohérent avec la forte proportion d’étudiants classés `bloque` ou `explorateur_chaotique`.

- **Tendance légèrement négative sur les premières tentatives** : la progression est négative sur les tentatives 1 à 40, ce qui peut indiquer que les premières soumissions s'éloignent parfois de la solution avant d'y revenir.
- **Variabilité croissante aux tentatives tardives** : à partir de la tentative 60, les valeurs deviennent plus erratiques, avec des pics positifs (jusqu'à +3) et négatifs marqués (jusqu'à -5 autour de la tentative 85). Cela reflète des comportements atypiques d'étudiants ayant soumis un très grand nombre de tentatives, probablement représentatifs des profils les plus en difficulté.
- **Pic négatif extrême à la tentative ≈ 85** : ce même étudiant responsable du pic de distance ZSS (Figure 4.2) semble également être à l'origine de cette chute brutale de progression, confirmant une restructuration massive et contre-productive du code à ce stade.

4.4 Synthèse

L'analyse des données AlgoPython révèle que la majorité des étudiants rencontre des difficultés à progresser de manière régulière et structurée. Les profils `bloque` et `explorateur_chaotique` représentent à eux seuls plus de 60% des cas.

Ces résultats sont d'autant plus significatifs qu'AlgoPython fournit déjà un feedback à chaque tentative. Le fait qu'une si grande proportion d'étudiants reste bloquée ou adopte une stratégie d'essais-erreurs non structurée, malgré ces retours immédiats, suggère que le feedback existant n'est pas toujours suffisant pour guider efficacement la correction du code.

Notre pipeline apporte ainsi une analyse complémentaire : plutôt que de fournir un feedback supplémentaire, il permet de caractériser *comment* les étudiants réagissent aux feedbacks existants, d'identifier les profils d'apprentissage qui en tirent le moins parti, et d'outiller les enseignants pour une intervention pédagogique ciblée.

Chapitre 5

Conclusion

5.1 Limites du projet

Malgré les résultats obtenus, plusieurs limites doivent être prises en compte.

Limites liées aux AST

L'analyse repose principalement sur la structure syntaxique des programmes via les AST. Cependant, deux programmes syntaxiquement proches peuvent produire des comportements très différents, et inversement.

De plus, certains codes étudiants sont syntaxiquement invalides, ce qui empêche parfois la génération correcte des arbres syntaxiques.

Limites de la distance de Zhang-Shasha

La distance de Zhang-Shasha mesure des différences structurelles mais ne capture pas complètement la sémantique des programmes.

Certaines modifications pédagogiquement importantes peuvent produire de faibles distances, tandis que de simples restructurations peuvent entraîner de fortes distances AST.

Limites de la classification

La classification des profils étudiants repose actuellement sur des heuristiques définies manuellement à partir de plusieurs indicateurs :

- nombre de tentatives ;
- taux de réussite ;
- distances AST ;
- progression vers la solution ;
- temps passé.

Ces profils ne résultent pas encore d'un apprentissage supervisé ou non supervisé. Ils constituent donc une première approximation des comportements étudiants.

5.2 Perspectives

Plusieurs pistes d'amélioration peuvent être envisagées afin d'étendre les capacités du projet.

- Extension du pipeline aux autres plateformes d'apprentissage comme SPY ou Pyrate ;
- Développement d'une interface de visualisation interactive ;
- Intégration de méthodes de clustering automatiques ;
- Génération automatique de feedback pédagogique à l'aide de modèles de langage ;
- Analyse plus fine des trajectoires d'apprentissage ;
- Détection automatique des stratégies de résolution.

Ces perspectives permettraient d'améliorer la précision des analyses et d'apporter des outils d'aide pédagogique plus avancés.

Bibliographie

- [1] Chris PIECH, Jonathan HUANG, Andy NGUYEN, Mike PHULGENCIO, Sherrie WANG, Leonidas GUIBAS et Mehran SAHAMI. « Learning Program Embeddings to Propagate Feedback on Student Code ». In : *Proceedings of the 32nd International Conference on Machine Learning (ICML)*. 2015, p. 1093-1102 (page 2).
- [2] Kaizhong ZHANG et Dennis SHASHA. « Simple Fast Algorithms for the Editing Distance between Trees and Related Problems ». In : *SIAM Journal on Computing* 18.6 (1989), p. 1245-1262. DOI : [10.1137/0218082](https://doi.org/10.1137/0218082) (page 3).
- [3] Elena L. GLASSMAN, Jeremy SCOTT, Rishabh SINGH, Philip J. GUO et Robert C. MILLER. « OverCode : Visualizing Variation in Student Solutions to Programming Problems at Scale ». In : *ACM Transactions on Computer-Human Interaction (TOCHI)*. T. 22. 2. 2015, p. 1-35. DOI : [10.1145/2699751](https://doi.org/10.1145/2699751) (page 3).
- [4] Thomas W. PRICE, Rui ZHI et Tiffany BARNES. « Factors Influencing Novice Programmers' Difficulty in Intermediate Code States ». In : *Proceedings of the 48th ACM Technical Symposium on Computer Science Education (SIGCSE)*. 2017, p. 117-122. DOI : [10.1145/3017680.3017724](https://doi.org/10.1145/3017680.3017724) (page 3).
- [5] Stuart RUSSELL et Peter NORVIG. *Artificial Intelligence : A Modern Approach*. 4^e éd. Hoboken, NJ : Pearson, 2020 (page 4).
- [6] Rishabh SINGH, Sumit GULWANI et Armando SOLAR-LEZAMA. « Automated Feedback Generation for Introductory Programming Assignments ». In : *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2013, p. 15-26. DOI : [10.1145/2491956.2462195](https://doi.org/10.1145/2491956.2462195) (page 4).

Annexe A

Cahier des charges

Sujet original : ai2d.hosted.lip6.fr/node/759

Encadrants : Sébastien Jolivet, Sébastien Lallé et Amel Yessad

Contexte

Ce projet s’inscrit dans l’étude de plusieurs environnements d’apprentissage de la programmation où les étudiants produisent plusieurs versions successives de code pour résoudre un problème. Les plateformes concernées sont AlgoPython, Pyrates et SPY.

Objectif principal

Proposer un système permettant d’identifier et de caractériser les évolutions entre le code initial et le code final soumis par les étudiants, en exprimant ces modifications de manière lisible et pertinente pour un enseignant d’informatique.

Outils

- Librairie [ast-error-detection](#) : convertit deux codes en AST et calcule les modifications nécessaires pour passer de l’un à l’autre.
- Bases de codes produits par des étudiants dans différents contextes (AlgoPython, Pyrates, SPY).

Objectifs fixés

1. Couche d’interprétation sémantique

Au lieu de simplement compter les modifications, le système doit pouvoir exprimer les changements en termes significatifs. Les solutions envisagées sont :

- utiliser la librairie [ast-error-detection](#) pour générer les AST des différentes versions de code et récupérer les opérations d’édition (insertions, suppressions, substitutions) entre deux soumissions successives ;

- définir une liste de catégories de modifications pédagogiquement parlantes : ajout/-suppression de structure de contrôle (`if`, `for`, `while`), ajout/suppression de variable, modification d’expression ou de valeur, restructuration de blocs (indentation, déplacement de code) ;
- concevoir des règles traduisant les opérations AST en ces catégories (par exemple : insertion d’un nœud `If` $\rightarrow$ « ajout d’une condition ») ;
- intégrer la fonction d’interprétation dans le pipeline existant (récupération JSON, construction AST, affichage du code) ;
- produire une sortie structurée (JSON) listant, pour chaque paire de codes, les modifications classées par catégorie, avec éventuellement un niveau de gravité ou d’impact.

2. Analyse des trajectoires

En prolongement, le projet propose d’identifier des trajectoires de programmation similaires et de classer les groupes de codes. Les solutions envisagées sont :

- filtrer les itérations de codes d’une même personne selon le problème (code vide, sans changement, etc.) ;
- comparer une instance à une autre et mettre en valeur les changements significatifs ;
- identifier les différentes catégories d’évolution des codes et les y classer ;
- représenter chaque trajectoire de l’étudiant (pour un exercice donné) comme une séquence d’états, chaque état étant décrit par un vecteur de caractéristiques : nombre de modifications, succès ou échec, etc. ;
- relier les profils à des informations pédagogiques (réussite de l’exercice, nombre total de soumissions, temps passé) pour proposer une interprétation utile aux enseignants.

3. Système de visualisation

Une interface permettant de visualiser l’évolution des codes de manière compréhensible. Les spécifications détaillées de ce module restent à déterminer.

Compétences techniques

- Bonne maîtrise de Python.
- Connaissances en analyse de données (nettoyage, exploration, préparation).
- Bases en algorithmique sur les arbres (parcours, comparaison d’arbres).

Annexe B

Manuel utilisateur

Le code source du projet est disponible sur GitHub : github.com/Driw0x/Projet-AI2D.

Installation

Cloner le dépôt puis installer les dépendances :

```
git clone https://github.com/Driw0x/Projet-AI2D.git
cd Projet-AI2D
pip install -r requirements.txt
```

Lancer le projet

Le fichier `main.py` constitue le point d'entrée principal du pipeline :

```
python main.py
```

Le pipeline effectue les opérations suivantes :

1. chargement des données AlgoPython ;
2. nettoyage et normalisation des données ;
3. génération des transitions entre tentatives ;
4. analyse AST et calcul des distances ;
5. détection des erreurs ;
6. génération des cas d'étude ;
7. classification des profils étudiants ;
8. génération des visualisations statistiques.

Certaines étapes peuvent être activées ou désactivées directement dans `main.py` afin de limiter le temps de calcul lors des expérimentations.